

Performance Evaluation of Hardening Measures for Quantum Key Management Systems

Jonas Berl^{(1),(2)}, Mario Wenning^{(1),(3)}, Helmut Grießer⁽¹⁾ and Tobias Fehenberger⁽¹⁾

⁽¹⁾ Adva Network Security GmbH, Munich, Germany, jonas.berl@advasecurity.com

⁽²⁾ Communications Engineering Lab, KIT, Karlsruhe, Germany

⁽³⁾ Chair of Communication Networks, TUM, Munich, Germany

Abstract We propose a hardened QKMS for resilient key relaying, combining secret sharing, PQC-hybridization, path verification, and confidential computing techniques. Emulation of a nation-wide QKD network demonstrates that these safeguards add only minor processing overhead, with overall key relay latency dominated by propagation delay. ©2026 The Author(s)

Introduction

Quantum key distribution (QKD) enables an information-theoretic secure key exchange between two authenticated parties [1]. With the increased maturity of QKD devices, the deployment of QKD networks (QKDNs) has gained momentum over the past years [2]. Since the reach of present-day QKD devices is limited, nodes out of range generate shared key material by relaying end-to-end (E2E) keys through trusted nodes (TNs) [3], orchestrated by a quantum key management system (QKMS). Because the E2E key material appears in plain text during the key relaying process, strict requirements on operational security are imposed on TNs. As a remedy, secret sharing [4, 5], path verification [6] and hybridization with post-quantum cryptography (PQC) [7] have been proposed to harden the key relaying.

In this paper, we present a novel QKMS that builds upon state-of-the-art hardening measures, namely secret sharing, path verification, and PQC-hybridization, and adds confidential computing techniques. This combination protects key material not only during transport across the QKDN, but also during processing within TNs. We further quantify the resulting overhead in an emulated nation-wide QKDN by comparing it with a traditional QKMS without hardening. Our analysis shows that the proposed security measures scale effectively to large networks and add only minor processing overhead.

Reducing the Trust in Trusted Nodes

To address the trust limitations of TNs, we exploit the path diversity of meshed optical networks by implementing the trivial (n, n) secret sharing scheme, in which the E2E key material is split across TN-disjoint key relay paths. With n being the threshold of the scheme, all shares are required to re-construct the E2E key material. Therefore, it remains secure as long as at least one key relay path is not compromised, i.e., one key relay path in which none of the TNs is under the

control of an attacker. For PQC-hybridization at the QKMS-layer, we transmit an additional secret share from the demand's source to the demand's destination over a PQC-secured E2E channel without intermediate relays, providing a fallback in case all key relay paths are compromised. Without countermeasures, the secret sharing scheme can become vulnerable to re-routing attacks, i.e., one or more compromised TNs manipulate the key relay path such that key relays are re-routed through malicious TNs [8]. This attack can be mitigated by path verification where source routing, i.e., the source node fixes the key relay path, is combined with a cryptographic proof included in the key relaying process that captures both the intended path and the actual TNs traversed. To protect the relayed key material not only in transport but also during processing, key relaying can be performed within a hardware-enforced trusted execution environment (TEE). This enables operators of QKDNs to cryptographically assess the trustworthiness of the underlying hardware of TNs and, if necessary, to prevent the processing of sensitive key material. Moreover, TEEs can protect the confidentiality of secrets, such as the relayed key material, while also ensuring the integrity of the executed code.

An overview of the hardening measures to reduce the impact of compromised TNs is depicted in Fig. 1. As shown for an exemplary demand from `src` to `dst`, there are two TN-disjoint paths of lengths l_1 and l_2 with l being the number of intermediate TNs, and there is a separate E2E path for the PQC-based key share (KS). At the source node, the first KS, referred to as $KS_{1/2}$, is relayed using the bit-wise XOR, denoted by $\oplus$, with the point-to-point (P2P) key material $k_{P2P}^{1,1}$. This operation is subsequently repeated for each TN in the key relay path and for each key relay path. At the destination node, all KSs are combined using XOR, i.e., the E2E key material is generated using $k_{E2E} = KS_{1/2} \oplus KS_{2/2} \oplus KS_{PQC}$. To cryptographically verify the key relay path intended by

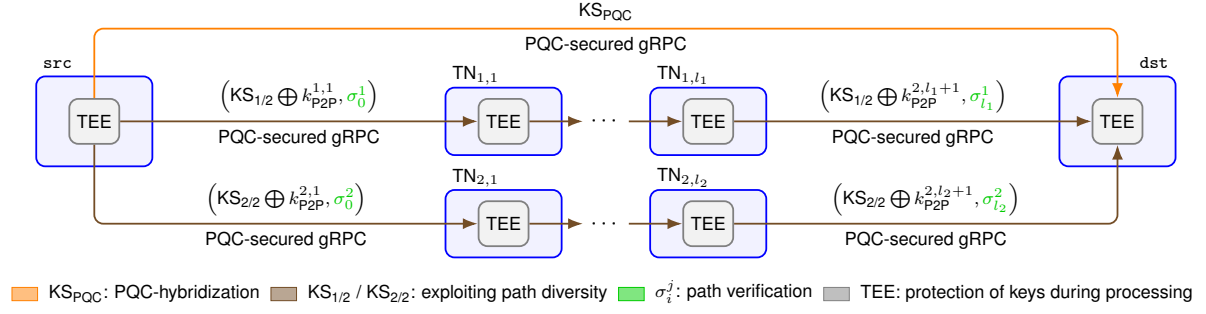


Fig. 1: Overview of implemented security measures to harden key relaying in QKMSs: PQC-hybridization using a separate E2E path, secret sharing over two TN-disjoint key relay paths, path verification and protection of key material during processing.

the source node and the order in which TNs are involved in the key relaying process, we define a chained signature scheme. Starting with an initial signature of the demand's source σ_0 , the TNs in key relay path j create chained signatures according to

$$\sigma_i^j \leftarrow \text{Sign}(sk_i^j, x_i^j \parallel \sigma_{i-1}^j) \quad i = \{1, \dots, l_j\},$$

where x_i^j is a data structure that contains the intended key relay path j and a binding to the KSs. Using a public-key cryptosystem, sk_i^j denotes the private key of node i in path j . To protect the key material during processing, key relaying is performed inside a TEE, which establishes E2E PQC-secured channels between two adjacent TNs in the key relay path or, for the PQC-based KS, directly between source and destination. This design ensures that sensitive secrets, including those required for the path verification scheme and the relayed key material, are never present in plaintext in the memory of the underlying hardware.

Experimental Setup

The nation-wide network topology considered in this study, *nobel-de* [9], is depicted in Fig. 2. For each link between two nodes, a pair of QKD devices is emulated that draws key material at a fluctuating key rate according to real-world monitoring data from a deployed QKDN. Note that, without loss of generality, we model any number of intermediate pairs of QKD devices between nodes as a single pair of devices. At each node, there is a key management entity (KME) that implements the key relaying. Following the layered architecture of QKDNs, the KME obtains P2P key material from QKD devices, performs key relaying over possibly multiple key relay paths, and provides the relayed key material to secure application entities, such as encryptors. The interface used for the retrieval and delivery of key material follows the ETSI GS QKD 014 specification [10]. For every QKD link, the KME in the TN maintains a key buffer, and the current buffer fill level is broadcast to other TNs through the OSPF protocol [11]. Using the topology information obtained via OSPF, each TN independently computes the key relay paths with

link costs inversely proportional to buffer fill levels. For non-standardized inter-KME communication, e.g., to coordinate the key retrieval from local QKD devices and for key relaying, we use a proprietary protocol based on gRPC [12] secured using TLSv1.3 with the hybrid key agreement scheme X25519MLKEM768 [13]. To cryptographically attest the key relay path through the QKDN, TNs generate and append chained signatures using the recently standardized scheme ML-DSA65 [14]. To protect the key material during processing, the KME is run within a TEE based on AMD SEV-SNP [15] on an AMD EPYC 9355 processor. Upon startup, all KMEs generate attestation reports to attest the underlying platform, including hardware-signed measurements such as the launch digest, i.e., a binding to the expected system state during startup, and platform information, e.g., the version of the trusted computing base.

In total, we use 44 virtual machines on a single host machine to emulate the QKDN, consisting of 17 KMEs, 26 QKD device pairs and a central monitoring node, which gathers extensive logging data and performance metrics. To mimic a real-world deployment, the inter-KME connections and the links between KMEs and QKD devices are realized with *virtio* bridges [16]. The fiber propagation between KMEs is emulated in the Linux kernel using *tc-netem* [17], assuming a propagation delay of 5.05×10^{-6} s/km. To assess the proposed hardening measures, we record monitoring data

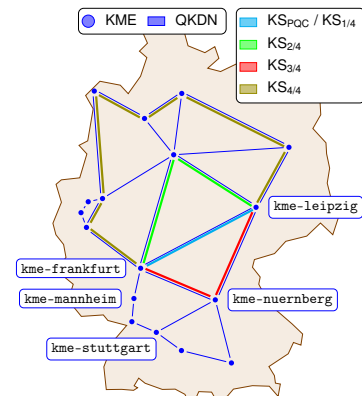


Fig. 2: *nobel-de* topology with the four TN-disjoint key relay paths for the demand *kme-leipzig* to *kme-frankfurt*.

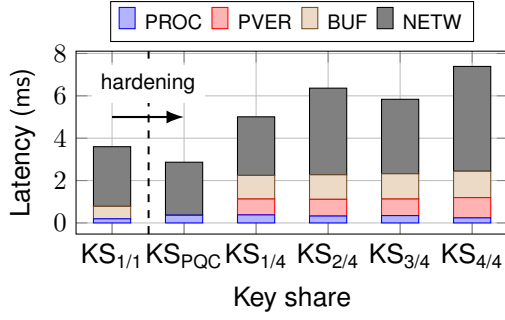


Fig. 3: Node-count normalized mean latency of the demand kme-leipzig to kme-frankfurt ($N_{\text{key}} = 16$).

over 30 minutes for four different configurations, i.e., any-to-any demands with $N_{\text{key}} = \{4, 8, 16, 32\}$ keys per key relay round. The rounds are initiated every 10 s after an initial startup phase of five minutes without key relays. Moreover, the proposed hardening measures can be disabled to emulate a traditional QKMS without hardening. In the hardened case, we use as many available TN-disjoint paths as possible.

Performance Analysis

With the implemented hardening measures, the latency associated with relaying KSs through a QKDN can be divided into four categories. First, networking-related latency (NETW) arises from the Linux networking stack on the TNs as well as from the propagation delay between the KMEs. Second, processing latency (PROC) is incurred during the creation and combination of KSs. Third, KMEs retrieve P2P key material from a local key buffer to decrypt incoming relayed key material, and to encrypt outgoing or forwarded key material (BUF). Fourth, the implemented path verification scheme introduces additional latency by generating, appending and verifying the chained signature scheme (PVER). In Fig. 3, we present the average latency contributions for the demand from kme-leipzig to kme-frankfurt, normalized to the number of nodes involved in key relaying. On the left side of the plot, we depict the latency of a QKMS with the same demands but without any hardening measures, while the right side shows the normalized latencies of the five KSs used to reconstruct the E2E key material in the hardened QKMS. We observe a normalized latency penalty of 1.4 ms from the traditional single-KS-relaying scheme to the first KS of the hardened QKMS, caused by the integration of path verification and confidential computing. Moreover, in the absence of hardening measures, the key buffer latency is lower because the smaller total number of relayed KSs reduces the load on the key buffers. Since KSs are pipelined, the overall latency is determined by the KS with the highest latency, which typically traverses the largest number of intermediate TNs. Therefore, PQC-hybridization comes with negligible overhead as the first KS already

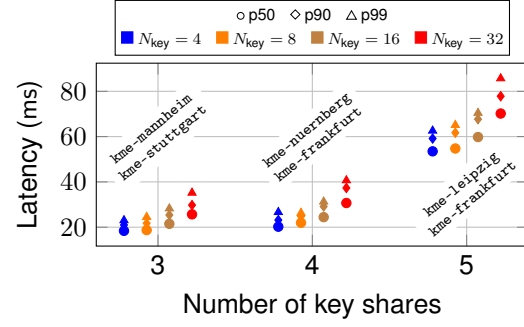


Fig. 4: Round-trip latencies for exemplary demands with three, four and five KSs.

exhibits a higher latency than the direct routed KS_{PQC}. As shown by the constant portion of key buffer and path verification latency for KS_{1/4} to KS_{4/4}, both scale linearly with path length and number of TNs. Overall, networking dominates the key relay latency, so that further optimizations of the hardening measures are likely to yield marginal reductions in latency only.

In Fig. 4, we present the round-trip latency of key relays, i.e., from source to destination and back, for exemplary demands with three, four, and five KSs as a function of N_{key} . Due to the stochastic nature of the emulation, we report the p50, p90 and p99 percentiles. As shown, increasing the number of relayed KSs from three to four incurs only a small penalty, provided that the longest key relay paths remain comparable. In contrast, as the fifth KS takes a long detour through the QKDN as shown in Fig. 2, we find that an increase from four to five causes a significant rise in key relay latency for the depicted demands. As the load increases with N_{key} , the key relay latency remains dominated by networking since latency differs more between the considered demands than within a demand. The comparably small gaps between the three percentiles, independent of load, indicate that the proposed hardening measures scale effectively. The investigated configuration with $N_{\text{key}} = 32$ relays 192 AES-256 keys per minute, while maintaining a p99 latency below 90 ms for the demand with the most KSs. Despite hardening, this setting enables up to 96 secure applications between any pair of nodes with a key renewal every 30 seconds.

Conclusion

In this work, we have shown that key relaying in QKDNs can be substantially strengthened through the combined use of secret sharing, PQC-hybridization, path verification, and confidential computing. Our emulation of a nation-wide QKDN reveals that the associated overhead remains small relative to propagation delay and that the approach scales to large networks with any-to-any demands. These findings demonstrate that the proposed hardening measures are both practical and security-essential, thereby establishing the state-of-the-art for secure operation of QKDNs.

Acknowledgments

This work is partially funded by the German Federal Ministry of Research, Technology and Space under project QuNET+ISQKMS (16KISQ104).

References

- [1] S. Pirandola *et al.*, “Advances in quantum cryptography”, *Advances in Optics and Photonics*, vol. 12, no. 4, pp. 1012–1236, 2020. DOI: 10.1364/AOP.361502.
- [2] H. Hübel *et al.*, “Deployed QKD networks in Europe”, in *2023 Optical Fiber Communications Conference and Exhibition (OFC)*, 2023.
- [3] B. Huttner *et al.*, “Long-range QKD without trusted nodes is not possible with current technology”, *npj Quantum Information*, vol. 8, no. 1, 2022. DOI: 10.1038/s41534-022-00613-4.
- [4] T. R. Beals and B. C. Sanders, “Distributed relay protocol for probabilistic information-theoretic security in a randomly-compromised network”, *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 5155 LNCS, pp. 29–39, 2008. DOI: 10.1007/978-3-540-85093-9_4.
- [5] L. Salvail, M. Peev, E. Diamanti, R. Alléaume, N. Lütkenhaus, and T. Länger, “Security of trusted repeater quantum key distribution networks”, *Journal of Computer Security*, vol. 18, no. 1, pp. 61–87, 2010.
- [6] J. Berl, M. Wenning, C. Mullan, H. Griebner, and T. Fehenberger, “Trust-enhanced quantum key management system for meshed QKD networks”, in *2025 Optical Fiber Communications Conference and Exhibition (OFC)*, 2025.
- [7] M. Geitz, R. Döring, and R.-P. Braun, “Hybrid QKD & PQC protocols implemented in the Berlin OpenQKD testbed”, in *2023 8th International Conference on Frontiers of Signal Processing (ICFSP)*, 2023.
- [8] P. James, S. Laschet, S. Ramacher, and L. Torresetti, “Key management systems for large-scale quantum key distribution networks”, in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, 2023.
- [9] *SNDlib*. [Online]. Available: <https://sndlib.put.poznan.pl/home.action> (visited on 04/15/2026).
- [10] *ETSI GS QKD 014 Protocol and data format of REST-based key delivery API*. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01_01_01_60/gs_qkd014v010101p.pdf (visited on 04/15/2026).
- [11] J. Moy, *OSPF Version 2*. [Online]. Available: <https://www.rfc-editor.org/info/rfc2328> (visited on 04/15/2026).
- [12] *gRPC: A high performance, open source universal RPC framework*. [Online]. Available: <https://grpc.io/docs/guides/auth/> (visited on 04/15/2026).
- [13] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, “Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3”, Internet Engineering Task Force, Internet-Draft, Feb. 2026. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/04/> (visited on 04/15/2026).
- [14] NIST, *Module-lattice-based digital signature standard*, 2024. DOI: 10.6028/NIST.FIPS.204.
- [15] J. Ménétreay *et al.*, “Attestation mechanisms for trusted execution environments demystified”, *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 13272 LNCS, pp. 95–113, 2022. DOI: 10.1007/978-3-031-16092-9_7.
- [16] R. Russell, “Virtio: Towards a de-facto standard for virtual I/O devices”, *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 95–103, Jul. 2008. DOI: 10.1145/1400097.1400108.
- [17] man7.org. “Tc-netem(8) - linux manual page”. (), [Online]. Available: <https://man7.org/linux/man-pages/man8/tc-netem.8.html> (visited on 04/15/2026).